

# Chapter 05: The Stuck Engineer's Diagnostic Decision Tree

---

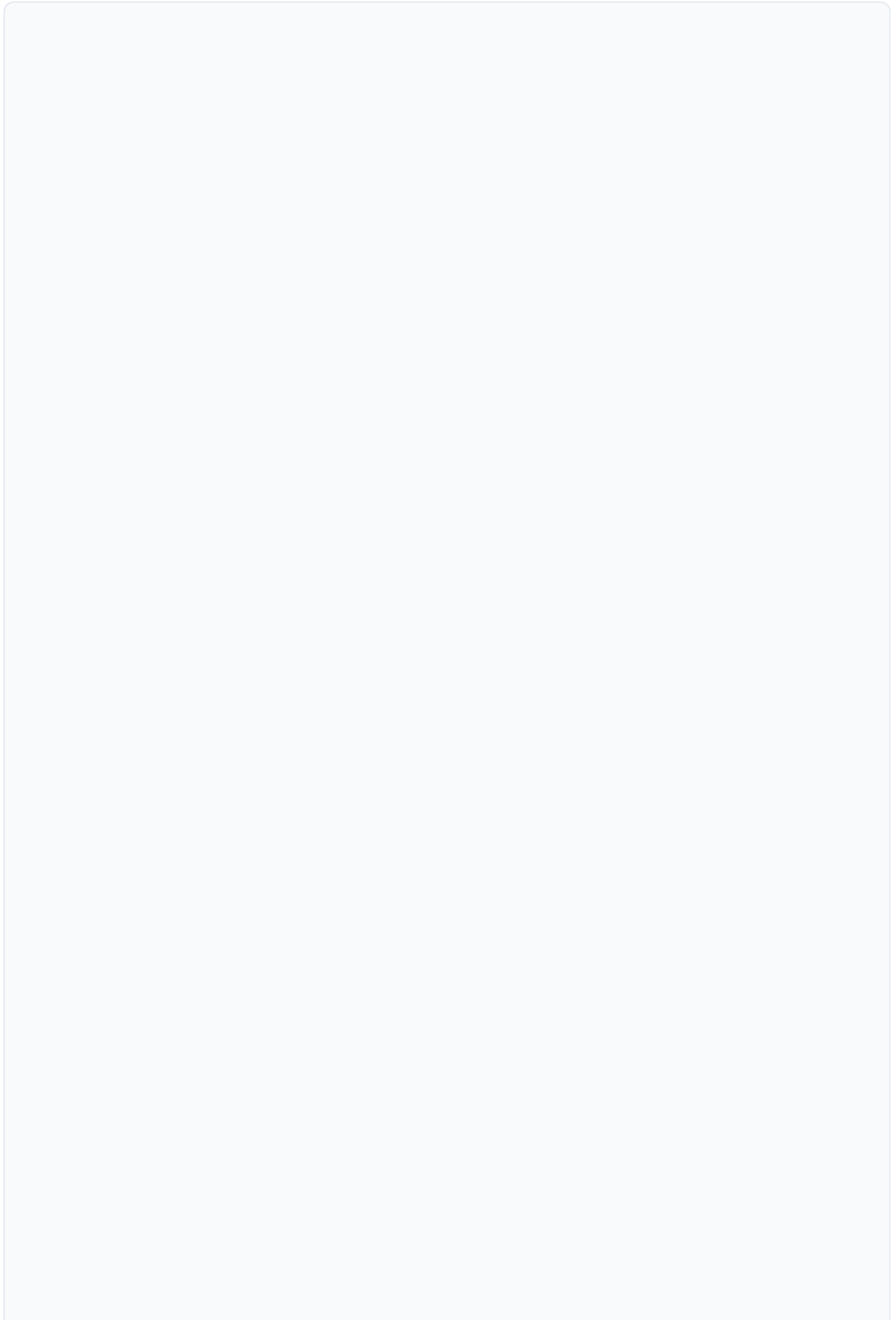
**The Blank Brain Phenomenon** Every engineer has experienced the nightmare: you read the problem description, your mind goes completely blank, and panic sets in. You have no idea what pattern applies, your heart rate spikes, and every second of silence feels like an eternity.

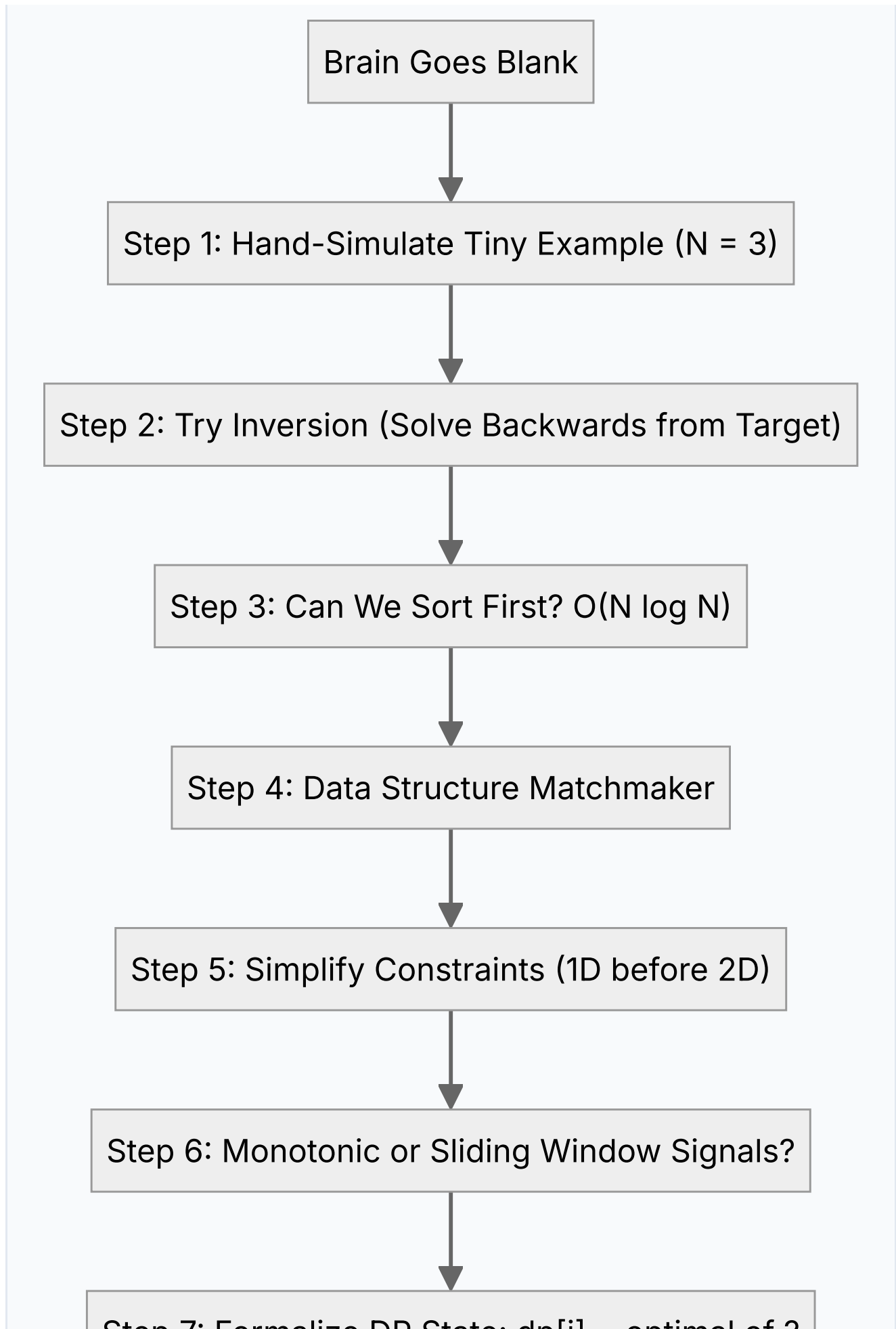
When intuition fails, you must switch to an **Algorithmic Diagnostic Tree**. This chapter provides a deterministic, 7-step triage system to unstick your brain and extract the optimal solution methodically.

---

## 1. The 7-Step Unsticking Decision Tree

---





## Step 7: Formalize DP State: $dp[i] = \text{optimal of ?}$

### Technique 1: Hand-Simulate a Tiny Example ( $N = 3$ )

When an array has 50 elements, patterns are obscured. Shrink the problem to  $N = 3$  or  $N = 4$  and manually trace your own subconscious logic on paper:

- *Ask yourself:* "How did my human brain know that 7 was the answer?"
- Did you scan left to right? Did you skip certain items? Whatever your human brain did intuitively can be turned into an algorithm!

### Technique 2: Inversion (Solve Backwards)

If finding a path from *Start*  $\rightarrow$  *Target* has an explosive branching factor of 5 choices per step, reverse the problem:

- Can you start at *Target* and work backwards towards *Start*?
- *Classic Example:* Jump Game. Checking if index 0 can reach index  $N - 1$  is messy. But starting at  $N - 1$  and checking if the previous index can reach it collapses the problem to a trivial greedy  $O(N)$  sweep!

### Technique 3: The Sorting Trade-Off ( $O(N \log N)$ )

If the array is unsorted and brute force is  $O(N^2)$ , ask:

*"Does the order of elements matter?"*

- If you are finding pairs, triplets, intervals, or frequencies, **order usually does not matter**.
- Sorting takes  $O(N \log N)$  and immediately unlocks **Binary Search** ( $O(\log N)$ ) or **Two Pointers** ( $O(N)$ )!

## 2. The Data Structure Matchmaker Matrix

When you can't think of an algorithm, match the problem's core requirement to its dedicated data structure:

| If the problem requires...                                          | The Dedicated Data Structure is...                        | Time Complexity                 |
|---------------------------------------------------------------------|-----------------------------------------------------------|---------------------------------|
| Tracking the <b>Top <math>K</math></b> largest or smallest elements | <b>Min-Heap / Max-Heap</b> ( <code>heapq</code> )         | $O(N \log K)$                   |
| Finding the <b>Next Greater / Smaller Element</b>                   | <b>Monotonic Stack</b>                                    | $O(N)$                          |
| Dynamic <b>Prefix / Autocomplete Search</b>                         | <b>Trie (Prefix Tree)</b>                                 | $O(L)$ where $L$ is word length |
| Checking <b>Connected Components / Cycle Detection</b>              | <b>Disjoint Set Union (Union-Find)</b>                    | $O(\alpha(N)) \approx O(1)$     |
| Finding the <b>Shortest Path in an Unweighted Graph</b>             | <b>Breadth-First Search (BFS)</b> with <code>deque</code> | $O(V + E)$                      |
| Finding the <b>Shortest Path in a Weighted Graph</b>                | <b>Dijkstra's Algorithm</b> with Priority Queue           | $O(E \log V)$                   |
| Fast <b>Range Sum Queries with Updates</b>                          | <b>Binary Indexed Tree (Fenwick) / Segment Tree</b>       | $O(\log N)$                     |
| Longest contiguous subarray satisfying a condition                  | <b>Sliding Window</b> (Two Pointers)                      | $O(N)$                          |

### 3. Dynamic Programming: The 3-Question Formulation

If a problem asks for "**Maximum**", "**Minimum**", "**Total Number of Ways**", or "**Is it possible to reach...**", it is almost always Dynamic Programming.

Stop trying to guess the whole code. Answer these 3 specific questions:

1. **What is the State?** What variables uniquely define where we are?

◦ *Example:* `dp(i, remaining_weight)`

2. **What is the Base Case?** What is the simplest possible input that has an obvious answer?

◦ *Example:* If `remaining_weight == 0`, return `0`. If `i == len(items)`, return `0`.

3. **What is the Transition Choice?** At state `i`, what are our possible actions?

◦ *Choice A:* Skip item `i`  $\rightarrow$  `dp(i + 1, remaining_weight)`

◦ *Choice B:* Take item `i`  $\rightarrow$  `value[i] + dp(i + 1, remaining_weight - weight[i])`

- *The Equation:* Take `max(Choice A, Choice B)`.

---

## 4. The Pre-Flight Edge Case Checklist

---

Before telling your interviewer your code is ready, run through this mental checklist:



Running through this checklist out loud demonstrates true principal-level engineering rigor.

## Further Reading

---

- [CP-Algorithms](#)
  - [Wikipedia: Algorithm design](#)
  - [Wikipedia: Greedy algorithm](#)
- 

## Check Yourself

---

Answer in your head or on paper first, then open each answer.

- ▼ 1. What question do you ask when you do not recognise a problem?

What is the input structure (array, graph, tree, string) and what is being optimised?

- ▼ 2. When do you suspect greedy versus DP?

Greedy when a local choice is provably safe; DP when choices interact through overlapping subproblems.

- ▼ 3. How do you decompose a hard problem?

Solve a smaller version, find the invariant, then generalise.

- ▼ 4. Why write down the brute force even if slow?

It gives a baseline and test oracle for the optimised version.